Colleges API: Endpoints

Base URL (local): `http://localhost:5050` **Base URL (deployed):** `https://postman-training-zkkv.vercel.app`
(colleges routes appear after you commit and redeploy)

For every request with a body, set the header `Content-Type: application/json`.

Summary

#	Method	Endpoint	Body	Success	Errors
1	GET	`/api/colleges`	none	200	none
2	GET	`/api/colleges/:id`	none	200	400, 404
3	POST	`/api/colleges`	one college object	201	400
4	PUT	`/api/colleges/:id`	one college object	200	400, 404
5	PUT (all)	`/api/colleges`	array of college objects	200	400
6	DELETE	`/api/colleges/:id`	none	200	400, 404
7	POST	`/api/colleges/reset`	none	200	none
8	PUT	`/api/colleges/:id/courses`	array of courses	200	400, 404
9	PATCH	`/api/colleges/:id/courses`	course object or array	200	400, 404
10	PUT	`/api/colleges/:id/fees`	array of fees	200	400, 404
11	PATCH	`/api/colleges/:id/fees`	fee object or array	200	400, 404

Full URLs

```
GET     {{baseUrl}}/api/colleges
GET     {{baseUrl}}/api/colleges?search=niet&state=Uttar Pradesh&city=Greater
Noida&type=PRIVATE&page=1&limit=10
GET     {{baseUrl}}/api/colleges/1
POST    {{baseUrl}}/api/colleges
PUT     {{baseUrl}}/api/colleges/1
PUT     {{baseUrl}}/api/colleges          (replaces ALL colleges)
DELETE  {{baseUrl}}/api/colleges/1
POST    {{baseUrl}}/api/colleges/reset
PUT     {{baseUrl}}/api/colleges/1/courses   (replaces the courses list)
PATCH   {{baseUrl}}/api/colleges/1/courses   (updates/adds courses by "code")
PUT     {{baseUrl}}/api/colleges/1/fees      (replaces the fees list)
PATCH   {{baseUrl}}/api/colleges/1/fees      (updates/adds fees by "courseCode")
```

Endpoint details

1. Get all colleges

`GET /api/colleges`

Optional query params: `search` (name or code), `state`, `city`, `type`, `page`, `limit` (max 100).

Response `200 OK`:

```
{ "total": 1, "page": 1, "limit": 100, "count": 1, "data": [ { "id": 1, "name": "...", "...": "..." } ]
}
```

2. Get one college

`GET /api/colleges/1`

- `200` returns the college.
- `400` if the id is not a positive integer.
- `404` if no college has that id.

3. Create a college

`POST /api/colleges`, body is one college object (see Dataset format).

Minimum valid body:

```
{ "name": "ABC College", "code": "ABC001" }
```

- `201 Created` returns the college with its new `id`, `createdAt`, `updatedAt`.
- `400` for a missing required field, an unknown key, a wrong type, or a duplicate `code`.

4. Replace one college

`PUT /api/colleges/1`, body is one full college object. `name` and `code` are required.

- `200` returns the updated college.
- `400` for validation errors or a code used by another college.
- `404` if the id doesn't exist.

5. Replace ALL colleges (put all)

`PUT /api/colleges`, body is a JSON array.

```
[
  { "name": "NIET", "code": "NIETGN133", "city": "Greater Noida" },
  { "name": "ABC College", "code": "ABC001" }
]
```

Response `200 OK`:

```
{ "message": "Replaced all colleges (2 total)", "total": 2, "data": [ ... ] }
```

- Ids are renumbered from 1. `[]` empties the dataset.
- `400` if the body is not an array, any item is invalid, or two items share a `code`. Errors are prefixed with the index, e.g. `[1] "name" is required...`.

6. Delete a college

`DELETE /api/colleges/1`

```
{ "message": "College 1 deleted", "deleted": { "id": 1, "name": "...", "...": "..." } }
```

- `404` if it doesn't exist or was already deleted.

7. Reset

`POST /api/colleges/reset` restores the single NIET sample record.

```
{ "message": "Colleges reset to sample data", "total": 1 }
```

8. Replace courses

`PUT /api/colleges/1/courses`, body is a JSON array. Each item needs a non-empty `code`.

```
[ { "name": "MBA", "code": "MBA", "durationYears": 2, "level": "POST_GRADUATE" } ]
```

- `200` returns the full updated college. The courses list is replaced entirely.
- `400` if the body is not an array or an item has no `code`. `404` if the college doesn't exist.

9. Update courses (partial)

`PATCH /api/colleges/1/courses`, body is one course object or an array of them, matched by `code`.

```
{ "code": "MBA", "durationYears": 3 }
```

- A matching course has the sent fields merged in; a `code` that doesn't exist yet is added.
- `200` returns the full updated college. `400` / `404` as above.

10. Replace fees

`PUT /api/colleges/1/fees`, body is a JSON array. Each item needs a non-empty `courseCode`.

```
[ { "courseCode": "MBA", "currency": "INR", "tuitionPerYear": 200000, "totalFee": 400000 } ]
```

- `200` returns the full updated college. The fees list is replaced entirely.
- `400` if the body is not an array or an item has no `courseCode`. `404` if the college doesn't exist.

11. Update fees (partial)

`PATCH /api/colleges/1/fees`, body is one fee object or an array of them, matched by `courseCode`.

```
{ "courseCode": "BTECH-CSE", "tuitionPerYear": 150000 }
```

- A matching fee has the sent fields merged in; a new `courseCode` is added.
- `200` returns the full updated college. `400` / `404` as above.

Dataset format

`name` and `code` are required non-empty strings. Every other key is optional; if omitted the server fills a default (`""`, `0`, `false`, `[]`, `null`, or an object of defaults). `code` must be unique (case-insensitive). Unknown top-level keys and wrong types return `400`. The server sets `id`, `createdAt`, `updatedAt` itself.

Key	Type
`name`	string, **required**
`code`	string, **required**, unique
`established`	number
`email`, `phone`, `address`, `city`, `state`, `country`, `website`, `videoUrl`, `logoUrl`	string
`ourRating`, `Placements`	number
`type`	string (e.g. `PRIVATE`)
`college_headLine`, `topCource`, `bannerImage`, `attendancePolicy`, `campusSize`, `collegeType`, `universityId`, `naacGrade`	string
`isHostel`, `isTopCollege`, `isVerifed`	boolean
`topCollegeRank`	any (null by default, not type-checked)
`about`, `highlights`, `streamTags`, `recognition`	array of strings
`reviews`, `CounselorDiscounts`, `images`, `testimonials`, `scholarships`, `cutoffs`	array
`courses`	array of `{ name, code, description, durationYears, approvedIntake, lateralEntry, nbaAccreditation, level }`
`fees`	array of `{ courseCode, currency, tuitionPerYear, otherChargesPerYear, totalFee }`
`placement`	object
`hostel`	object
`facilities`	object
`culture`	object
`ranking`	object
`admission`	object

Key	Type
`faculty`	object
`faqs`	array of `{ question, answer }`

Keys seeded as `null` (e.g. `approvedIntake`, `averagePackage`, `nirfRank`) accept any value. The nested objects are not checked key by key, only that they are objects.

Nested object shapes

```
{
  "placement": { "averagePackage": null, "highestPackage": null, "medianPackage": null, "placementRate": null, "recruiters": [], "recruiterLogos": [], "topRecruiters": [] },
  "hostel": { "isAvailable": true, "hostelCostYear": null, "messCostYear": null, "foodQuality": null, "roomTypes": [], "amenities": [] },
  "facilities": { "hasMedical": true, "hasTransport": true, "hasWifi": true, "hasSports": true, "hasLibrary": true, "hasCanteen": true, "hasGym": true, "nearestHospital": "", "transportDetails": "", "otherFacilities": [] },
  "culture": { "clubsCount": null, "clubCategories": [], "annualFests": [], "hasHackathons": true, "hasResearch": true, "hasCompetitions": true, "hackathonDetails": "", "researchDetails": "", "competitionDetails": "" },
  "ranking": { "nirfRank": null, "nirfCategory": "", "nirfYear": null, "qsRank": null, "qsYear": null, "naacGrade": "A", "naacScore": 3.23, "naacYear": null, "nbaPrograms": [], "aicteApproved": true, "ugcRecognized": true, "govtApprovals": [], "otherRankings": "" },
  "admission": { "entranceExams": [], "eligibility": "", "applicationProcess": "", "selectionProcess": "", "applicationFee": null, "applicationStart": null, "applicationEnd": null, "resultDate": null, "admissionStart": null, "importantDates": "", "websiteLink": "" },
  "faculty": { "totalFaculty": null, "studentRatio": null, "phdFacultyPct": null, "notableFaculty": [], "researchPapers": null, "patents": null }
}
```

Full example body (POST / PUT)

The complete NIET record with all 8 courses is in server/colleges-seed.json. Paste it into Body → raw → JSON.

Error responses

Validation (`400`):

```
{ "error": "Validation Failed", "details": ["\"name\" is required and must be a non-empty string", "Unknown field(s): bogus"] }
```

Duplicate code (`400`):

```
{ "error": "Validation Failed", "details": ["A college with code \"X1\" already exists"] }
```

Not found (`404`):

```
{ "error": "Not Found", "message": "College with id 99 not found" }
```

Status codes used

Code	Meaning
`200 OK`	Request succeeded
`201 Created`	New college created (POST)
`400 Bad Request`	Invalid body, bad JSON, invalid id, duplicate code
`404 Not Found`	College or route doesn't exist